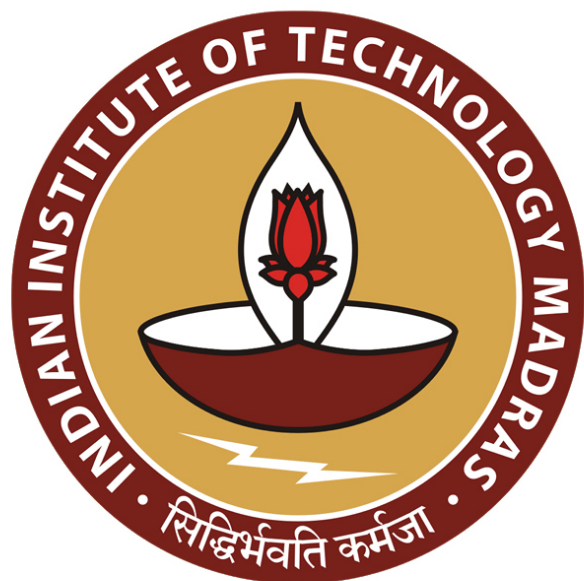




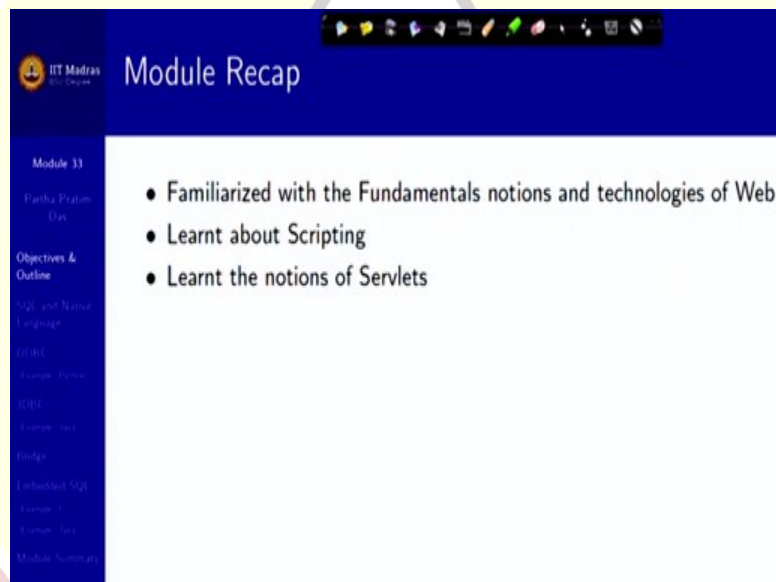
# **IIT Madras**

**ONLINE DEGREE**

**Database Management Systems**  
**Professor Partha Pratim Das**  
**Department of Computer Science and Engineering,**  
**Indian Institute of Technology, Kharagpur**  
**Application Design and Development/3: SQL and Native Language**

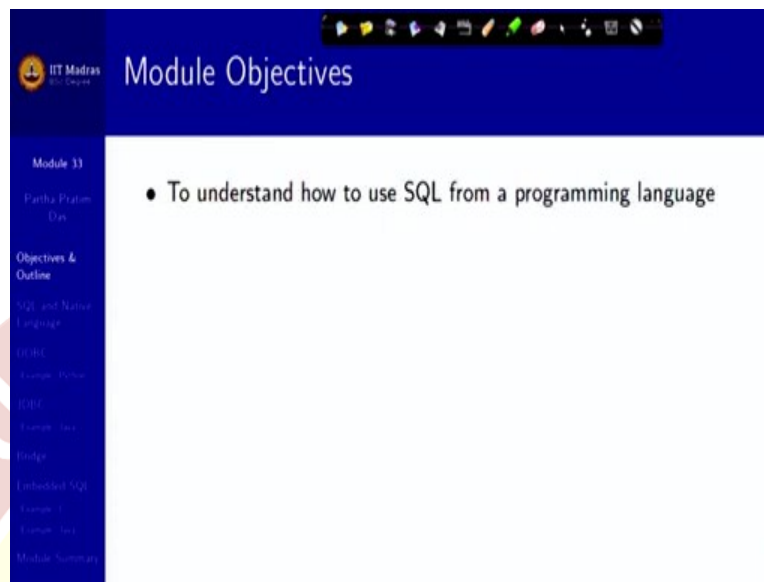
Welcome to Module 33 of Database Management Systems course in IIT Madras online B.Sc. program. We are discussing application design and development using databases in the back. We have discussed about the architecture. And since significant majority of these applications are web-based, Internet-based, we have also discussed about web fundamentals, the basic structure of the web architecture, the scripting and all these in the last module.

(Refer Slide Time: 01:00)



This is what you have learned.

(Refer Slide Time: 01:04)



Now, going forward, so if I just to give you a quick reminder of the three-tier architecture, there is a presentation layer which is interacting with the user it is on the browser. So, what is the requirement, development requirement there? You have to be able to write the, design the HTML the elements and all that and the validations or auto fill and so on, you need to have the JavaScript's or some client-side scripting for it. So, this is something which we can practice more in the application development course.

The second part is a middle tier which is an application or business logic layer where primarily you will have different kinds of logic in typical object-oriented design Java or C++ and you will have object models corresponding to the relational models and so how to do that you know from your programming, an object-oriented programming background, so that I assume that you will be able to do that.

To facilitate the creation of HTML pages from servicing the query we have servlets or server-side scripting JSP and so on, so which can help you actually more easily write the query response by filling up an HTML template with the query results. And we have taken a quick look at how to do that. This is not difficult. Though you have not done it, you can easily pick it up. So, that part the server-side scripting we can assume that you will be able to do.

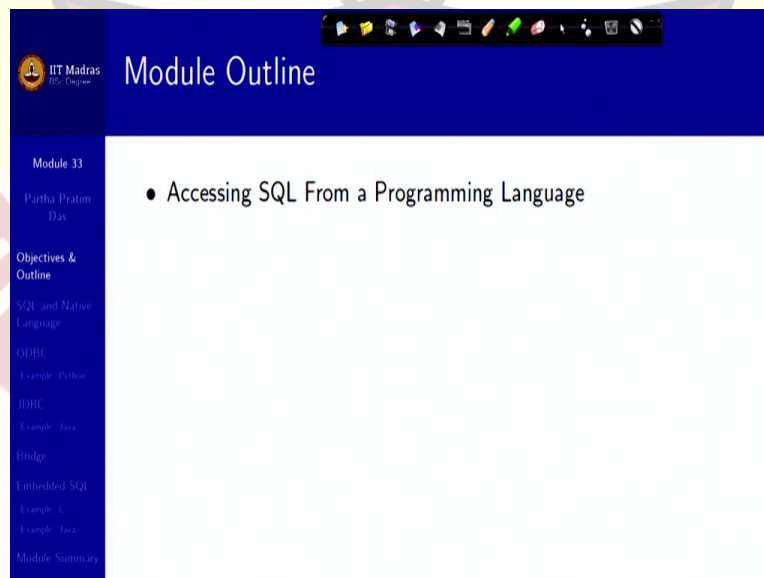
What is left? And beyond that is the data access layer or data layer, where you have the database, the tables, the query the constraints, the data instances and you know SQL to be able to access or

update those data. So, this is the whole paradigm. So, the only missing piece in this is the fact that in your servlet or in your high-level programming in the application layer based on the response, based on the request that you have got, you will need to get the right result from the database.

Now, database understands SQL. So, one part would be that you will have to go to the database and present the SQL query. If you present the SQL query, the database will give you a response. That response what is the response is a relation, which means it is a table. So that table you will have to take and put into HTML. So, anything from the beginning of the frontend to the point where you have the servlets or the high-level programming language, knowing what data it needs to extract from the database, this part is known to you. Once the query goes to the database, you know how to answer it. And once the table comes back again you know how to do the HTML and set it back.

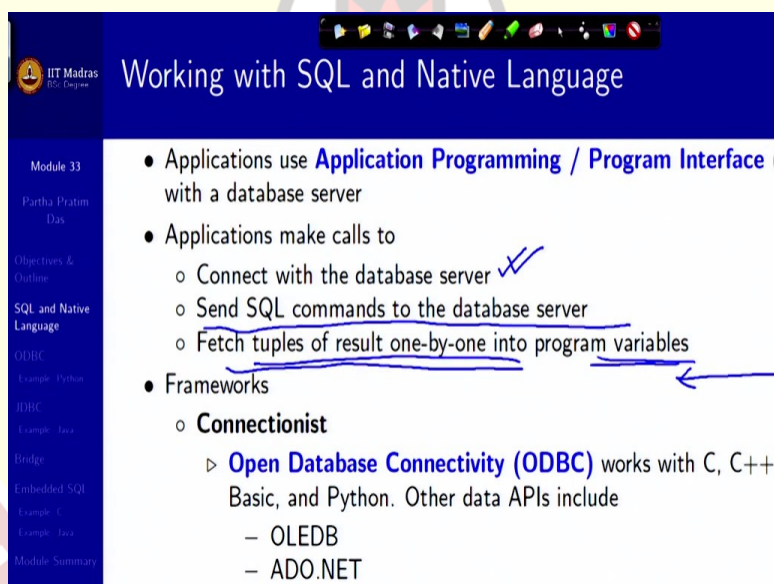
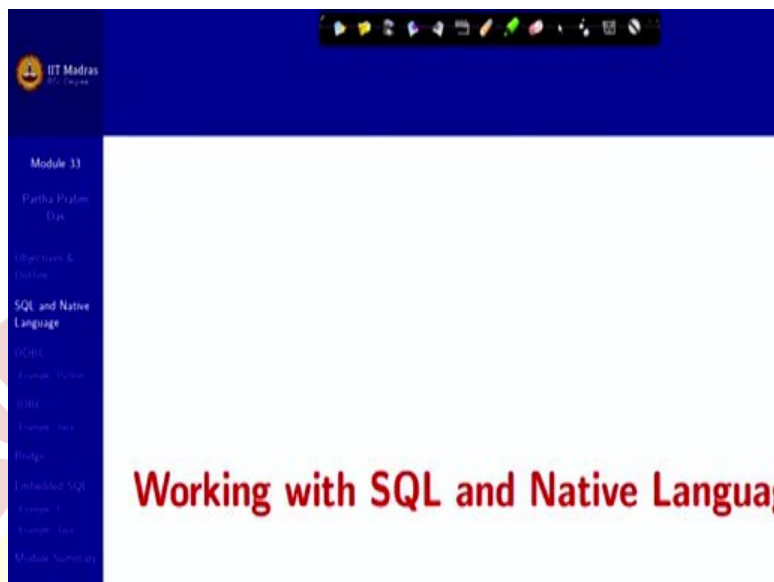
So, the missing part in the game, in the jigsaw is from the high level how do I make that SQL execute and get the result and how do I get that result back from the database into the high-level program.

(Refer Slide Time: 05:10)



That is what is the, that is a core application development area from the database perspective and we would like to discuss that working with SQL, sequel and native languages, high level programming languages.

(Refer Slide Time: 05:19)



**Working with SQL and Native Language**

- Applications use **Application Programming / Program Interface** with a database server
- Applications make calls to
  - Connect with the database server
  - Send SQL commands to the database server
  - Fetch tuples of result one-by-one into program variables
- Frameworks
  - **Connectionist** ✓
    - ▷ **Open Database Connectivity (ODBC)** works with C, C++, Basic, and Python. Other data APIs include
      - OLEDB ||
      - ADO.NET ||

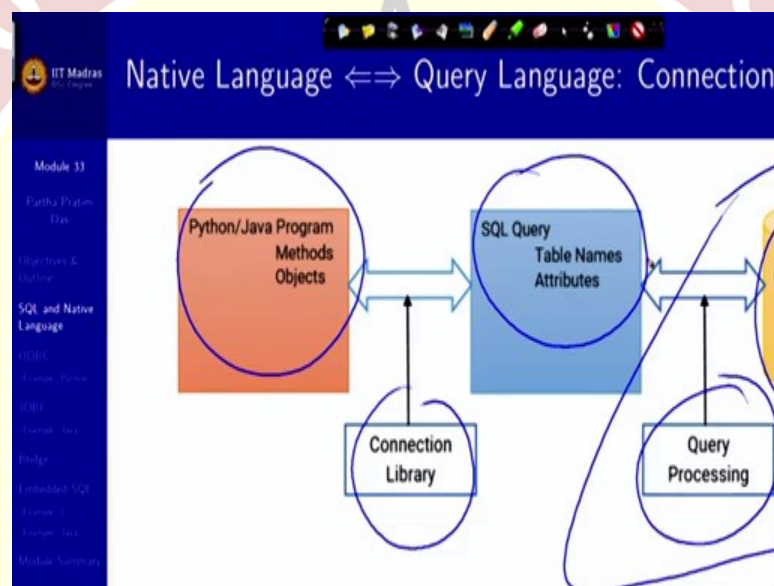
So, the basic idea is simple. Is, you will have to create some kind of a Application Programming Interface API, which it can interact with the database server. Now, what are the things that it grossly needs to do? It needs to connect with the database server. Because please do not assume that there is only one database server there could be multiple. So, it needs to connect to the database server. Even one database server will have multiple users with different rights and so on. So, you need to connect to the database server with the proper user credentials.

Once that is done, you need to send the SQL command in some way to the database server. Once you send that the database server knows how to process that SQL command to generate the data. It generates a table or you can say a sequence of tuples. And then you will have to fetch those tuples one by one because there is a sequence that stable rows different into your high-level program so that you can proceed further. And after you are totally done, what you can put here is close connection. Once you are done, you can settle close the connection so that it cannot be used by mistake somewhere else.

Now, for this there are two kinds of dominant framework one which is the connectionist framework which creates, I mean, kind of creates a driver which connects between the native language and the database server. This is called ODBC Open Database Connectivity and it works with almost all major programming languages. There are other connectionist APIs also which include say OLEDB, Object Linking and Embedding.

This Microsoft brought it in very early days. And ADO NET .NET and so on. And the other which is very popular is JDBC which is Java specific, which is Java Database Connectivity. So, this is one approach connectionist. So, in connectionist approach your SQL program is kind of created, executed through a set of APIs that these drivers provide. The other approach is embedding. In embedding, you put SQL commands in some format inside your native language, inside your native language program. This again works with a large number of programming languages. So, we will take a look at both of them one by one.

(Refer Slide Time: 08:50)



So, this is the overall architecture of the connectionist one. So, you have a high-level program and you have SQL query with tables and all that and you have this connection library, which could be ODBC, it be JDBC some set of APIs which allows you from the high-level language to submit the query, connect and submit the query and get the result. And then it is the inside the database process the typical SQL query processing using the database in schema this part is what we have already covered.

(Refer Slide Time: 09:33)

**ODBC**

- **Open Database Connectivity (ODBC)** is a standard API for accessing databases
- It aimed to be independent of database systems and operating system
- An application written using ODBC can be ported to other platforms client and server side, with few changes to the data access code
- ODBC is
  - A standard for application program to communicate with a database
  - An application program interface (API) to
    - ▷ Open a connection with a database
    - ▷ Send queries and updates
    - ▷ Get back results
- Applications such as GUI, Spreadsheets, etc. can use ODBC

So, given this, what is ODBC? It is a standard API for accessing database. So, the objective of ODBC is it is database system independent as well as it is operating system independent. That is, if you have written an application program using the ODBC APIs, then it will work on, on operating system and it will work for all database systems, which is very much required, otherwise migration would be a big problem. It can be ported to other platforms, both on the client as well as servers side with only few changes that may be required.

Those are required, few changes are required at times because of the hardware differences and so on. So, this is thematically what you will do in a ODBC used application, open the connection, send query and updates, get back result. Historically, it was developed by Microsoft and Simba Technology. And later on SQL is in 90s SQL subsumed it and it is in the standard.

(Refer Slide Time: 10:47)

 IIT Madras  
BSc. Computer

## ODBC (2): Python Example

Module 33  
Partha Pratim Das  
Objectives & Outline  
SQL and Native Language  
ODBC  
Example: Python  
JDBC  
Example: Java  
Bridge  
Embedded SQL  
Example: C  
Example: Java  
Module Summary

- The code uses a data source named "SQLS" from the odbc.ini file to connect and issue a query.
- It creates a table, inserts data using literal and parameterized statements and fetches the data

```
import pyodbc

conn = pyodbc.connect('DSN=SQLS;UID=test')
cursor=conn.cursor()
cursor.execute("create table rvtest (col3 varchar(10))")
cursor.execute("insert into rvtest value \"ABC\"")
cursor.execute("select * from rvtest")

while True:
    row=cursor.fetchone()
    if not row:
        break
    print(row)

cursor.execute("delete from rvtest")
cursor.execute("insert into rvtest value 20.0, 'XYZ'")
cursor.execute("select * from rvtest")

while True:
    row=cursor.fetchone()
```

 IIT Madras  
BSc. Computer

## ODBC (2): Python Example

Module 33  
Partha Pratim Das  
Objectives & Outline  
SQL and Native Language  
ODBC  
Example: Python  
JDBC  
Example: Java  
Bridge  
Embedded SQL  
Example: C  
Example: Java  
Module Summary

- The code uses a data source named "SQLS" from the odbc.ini file to connect and issue a query.
- It creates a table, inserts data using literal and parameterized statements and fetches the data

```
import pyodbc

conn = pyodbc.connect('DSN=SQLS;UID=test')
cursor=conn.cursor()
cursor.execute("create table rvtest (col3 varchar(10))")
cursor.execute("insert into rvtest value \"ABC\"")
cursor.execute("select * from rvtest")

while True:
    row=cursor.fetchone()
    if not row:
        break
    print(row)

cursor.execute("delete from rvtest")
cursor.execute("insert into rvtest value 20.0, 'XYZ'")
cursor.execute("select * from rvtest")

while True:
    row=cursor.fetchone()
```

## ODBC (2): Python Example

### Module 33

Partha Pratim Das

Objectives & Outline

SQL and Native Language

ODBC

Example: Python

Example: Java

Bridge

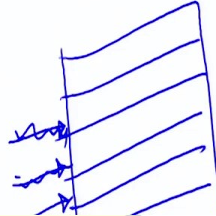
Embedded SQL

Example: C

Example: Java

Module Summary

- The code uses a data source named "SQLS" from the odbc.ini file to connect and issue a query.
- It creates a table, inserts data using literal and parameterized statements and fetches the data



```
import pyodbc

conn = pyodbc.connect('DSN=SQLS;UID=test')
cursor=conn.cursor()
cursor.execute("create table rvtest (col1
col2 varchar(10))")
cursor.execute("insert into rvtest value
\"ABC\"")
cursor.execute("select * from rvtest")

while True:
    row=cursor.fetchone()
    if not row:
        break
    print(row)

cursor.execute("delete from rvtest")
cursor.execute("insert into rvtest value
20.0, 'XYZ'")
cursor.execute("select * from rvtest")

while True:
    row=cursor.fetchone()
```

## ODBC (2): Python Example

### Module 33

Partha Pratim Das

Objectives & Outline

SQL and Native Language

ODBC

Example: Python

Example: Java

Bridge

Embedded SQL

Example: C

Example: Java

Module Summary

- The code uses a data source named "SQLS" from the odbc.ini file to connect and issue a query.
- It creates a table, inserts data using literal and parameterized statements and fetches the data

```
import pyodbc

conn = pyodbc.connect('DSN=SQLS;UID=test')
cursor=conn.cursor()
cursor.execute("create table rvtest (col1
col2 varchar(10))")
cursor.execute("insert into rvtest value
\"ABC\"")
cursor.execute("select * from rvtest")

while True:
    row=cursor.fetchone()
    if not row:
        break
    print(row)

cursor.execute("delete from rvtest")
cursor.execute("insert into rvtest value
20.0, 'XYZ'")
cursor.execute("select * from rvtest")

while True:
    row=cursor.fetchone()
```

सिद्धिर्भवति कर्मजा



## ODBC (2): Python Example

Module 33

Partha Pratim Das

Objectives & Outline

SQL and Native Language

ODBC

**Example: Python**

JDBC

Example: Java

Bridge

Embedded SQL

Example: C

Example: Java

Module Summary

- The code uses a data source named "SQLS" from the odbc.ini file to connect and issue a query.
- It creates a table, inserts data using literal and parameterized statements and fetches the data

```
import pyodbc

conn = pyodbc.connect('DSN=SQLS;UID=test
cursor=conn.cursor()
cursor.execute("create table rvtest (col
col3 varchar(10))")
cursor.execute("insert into rvtest value
\"ABC\")")
cursor.execute("select * from rvtest")

while True:
    row=cursor.fetchone()
    if not row:
        break
    print(row)

cursor.execute("delete from rvtest")
cursor.execute("insert into rvtest value
20.0, 'XYZ')
cursor.execute("select * from rvtest")

while True:
    row=cursor.fetchone()
```



## ODBC (2): Python Example

Module 33

Partha Pratim Das

Objectives & Outline

SQL and Native Language

ODBC

**Example: Python**

JDBC

Example: Java

Bridge

Embedded SQL

Example: C

Example: Java

Module Summary

- The code uses a data source named "SQLS" from the odbc.ini file to connect and issue a query.
- It creates a table, inserts data using literal and parameterized statements and fetches the data

```
import pyodbc

conn = pyodbc.connect('DSN=SQLS;UID=test
cursor=conn.cursor()
cursor.execute("create table rvtest (col
col3 varchar(10))")
cursor.execute("insert into rvtest value
\"ABC\")")
cursor.execute("select * from rvtest")

while True:
    row=cursor.fetchone()
    if not row:
        break
    print(row)

cursor.execute("delete from rvtest")
cursor.execute("insert into rvtest value
20.0, 'XYZ')
cursor.execute("select * from rvtest")

while True:
    row=cursor.fetchone()
```



**Module 33**

Partha Pratim Das

Objectives & Outline

SQL and Native Language

ODBC

**Example: Python**

JDBC

Example: Java

Bridge

Embedded SQL

Example: C

Example: Java

Module Summary

## ODBC (2): Python Example

- The code uses a data source named "SQLS" from the odbc.ini file to connect and issue a query.
- It creates a table, inserts data using literal and parameterized statements and fetches the data

```
import pyodbc

conn = pyodbc.connect('DSN=SQLS;UID=test;PWD=test')
cursor = conn.cursor()

cursor.execute("create table rvtest (col1 int, col2 varchar(10))")
cursor.execute("insert into rvtest values (1, 'ABC')")
cursor.execute("select * from rvtest")

while True:
    row = cursor.fetchone()
    if not row:
        break
    print(row)

cursor.execute("delete from rvtest")
cursor.execute("insert into rvtest values (20.0, 'XYZ')")
cursor.execute("select * from rvtest")

while True:
    row = cursor.fetchone()
    if not row:
        break
    print(row)
```

So, let me just quickly show you an ODBC example with Python. So, Python has a ODBC library called pyodbc. Python has several ways we will see. So, the first thing you do is this library.connect. So, this is an API. And you pass as parameter the different information that is required, which is the database ID, SQLS, the user ID and the password. You will require that. Once you have done that, then you have got a connection object.

On the connection object, you create a cursor. A cursor is nothing but it is like pointer, like if you have a table, say you have a table with several rows, a cursor is a pointer, which is somewhere in the table and then it moves to the next. If you say I am done with this row, it goes to the next. You are done with this row goes to the next. So, it is like a pointer or you can think of it in C++, Java style, you can think of it as an iterator. It basically is an iterator. So you create a cursor and then on that cursor you execute. So, execute is basically it will take an SQL command and execute.

So, you can see that we have given a command to create table simply as a text string. So, this is an SQL command, we just give it a text string. Then you have another SQL command. So, it creates a table, inserts the data and there is a third statement executed which is select. When you do that, it will get back a result. Select star, so it will get back all those which is currently only one row, because you have created and inserted only one, will get back that. So, where will you get it, you will get it in the fetchone method of the cursor. So, fetchone will get you one row at a

time. So, it gets you take it in the row. If it is not a row, which means that if it is at the end, then you break, you are done. Otherwise, you print. Similarly, it shows few more deletions inside.

I mean you can write any kind of SQL query in this manner. This is your connect part, this is your result part and this is your SQL query part. Very simple structure. You can write, these are variants of the particular pyodbc style. You can directly write everything in the string or you can write the query as a string and keep some parameters which you put later on. So, that is the basic ODBC structure.

(Refer Slide Time: 13:52)

**JDBC**

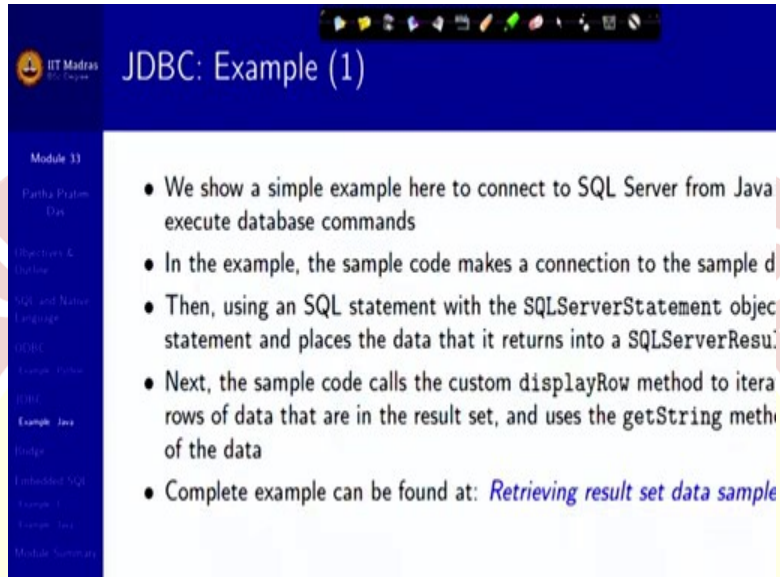
- **Java Database Connectivity (JDBC)** is an API for the program which defines how a client may access a database
- It is a Java-based data access technology used for Java database connection
- JDBC supports a variety of features for querying and updating data, query results; metadata retrieval, such as querying about relations present in the database and the names and types of relation attributes
- Model for communicating with the database:
  - Open a connection ✓
  - Create a "statement" object ✓
  - Execute queries using the Statement object to send queries and fetch results ✓
  - Exception mechanism to handle errors ✓
- JDBC, originally released by Sun Microsystems released as part of Java 1.2

Similarly, there is a API for Java called JDBC. And this was brought out much later after ODBC in 97 in the Java Development Kit 1.1 and since then it has been the part of the Java Standard Edition platform supported by Oracle, of course, and it specifically works with Java. And since Java is a significantly platform independent language and efficient in terms of writing application programs. So, JDBC has that a very good acceptance.

Again, the steps are like ODBC open the connection. This is which is specific to JDBC because JDBC is completely object based, because it is specifically for Java. ODBC cannot assume that because ODBC supports languages which do not have any object. So, here it creates a statement object that becomes the handle through which you do everything and you execute the query using this statement objects to send query, to fetch results, the statement is your handle to do

things, and it also has exception mechanism to handle errors very efficient, very simple and nice to use.

(Refer Slide Time: 15:24)

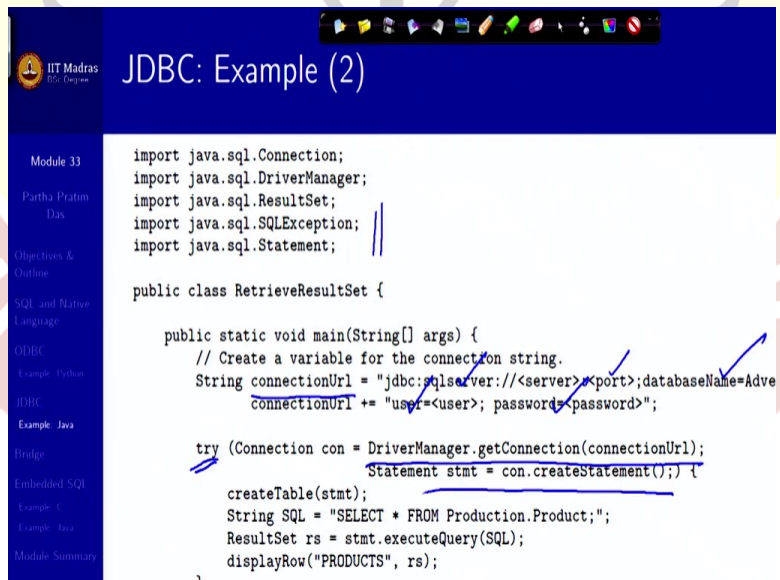


**JDBC: Example (1)**

- We show a simple example here to connect to SQL Server from Java execute database commands
- In the example, the sample code makes a connection to the sample d
- Then, using an SQL statement with the `SQLServerStatement` object and places the data that it returns into a `SQLServerResu`.
- Next, the sample code calls the custom `displayRow` method to iterate rows of data that are in the result set, and uses the `getString` method of the data
- Complete example can be found at: [Retrieving result set data sample](#)

So, here, I just show a simple example, which is a Java code using the JDBC.

(Refer Slide Time: 15:33)



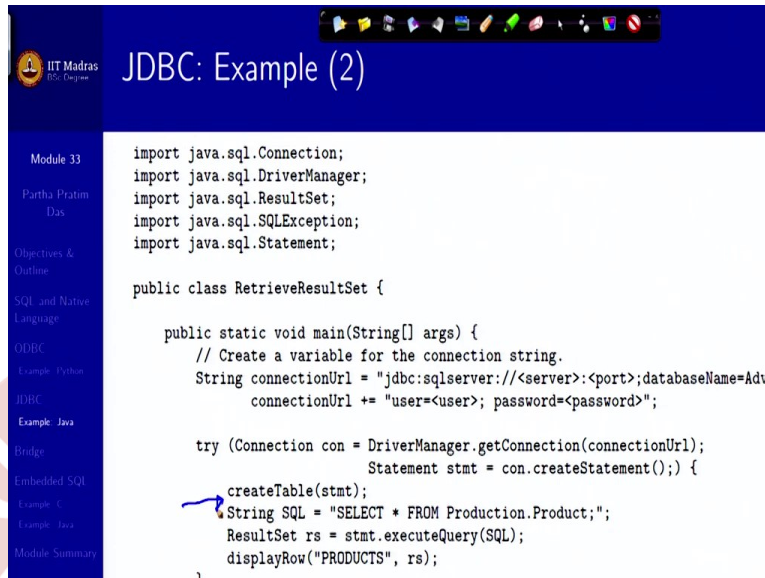
**JDBC: Example (2)**

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException; //
import java.sql.Statement;

public class RetrieveResultSet {

    public static void main(String[] args) {
        // Create a variable for the connection string.
        String connectionUrl = "jdbc:sqlserver://<server>:<port>;databaseName=Adventureworks;
        connectionUrl += "user=<user>; password=<password>";

        try (Connection con = DriverManager.getConnection(connectionUrl);
            Statement stmt = con.createStatement();) {
            createTable(stmt);
            String SQL = "SELECT * FROM Production.Product;";
            ResultSet rs = stmt.executeQuery(SQL);
            displayRow("PRODUCTS", rs);
        }
    }
}
```



**JDBC: Example (2)**

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class RetrieveResultSet {

    public static void main(String[] args) {
        // Create a variable for the connection string.
        String connectionUrl = "jdbc:sqlserver://<server>:<port>;databaseName=Adve
            connectionUrl += "user=<user>; password=<password>";

        try (Connection con = DriverManager.getConnection(connectionUrl);
            Statement stmt = con.createStatement();) {
            createTable(stmt);
            String SQL = "SELECT * FROM Production.Product;";
            ResultSet rs = stmt.executeQuery(SQL);
            displayRow("PRODUCTS", rs);
        }
    }
}
```

Now, initially, here, you will see that certain imports are making which is JDBC related imports. The first thing is connection. So, I need to build the connection URL. What do I connect to. I need the server with the port, where it is, I need the database in that, name of the database, I need the user, I need the password. So, I put all of this into a string and then I through my driver manager, because I said JDBC is a driver, we say get connection passing this parameter. So, it gives me connection. And with that connection, I do a create statement.

The first thing was to get connected and then have a statement which will work as a handle between Java and the database. There is a try to take care of if the connection could not be done, for example, the password is wrong or the database name does not exist and so on. If it is then you handle it here with the stacktrace you print. If it is correct, then I first step I do is with the statement I called create table. So, let us see what is create table. As the name suggests it will create a table. So, let us go to the next slide and see what is create table.

(Refer Slide Time: 17:11)

 IIT Madras  
RSC Campus

### JDBC: Example (3)

**Module 33**  
Partha Pratim Das  
Objectives & Outline  
SQL and Native Language  
ODBC  
Example: Python  
JDBC  
**Example: Java**  
Bridge  
Embedded SQL  
Example: C  
Example: Java  
Module Summary

```
private static void displayRow(String title, ResultSet rs) throws SQLException {
    System.out.println(title);
    while (rs.next()) { // Iterate on Table("ProductID", "Name")
        System.out.println(rs.getString("ProductID") + " : " + rs.getString("Name"));
    }
}

private static void createTable(Statement stmt) throws SQLException {
    stmt.execute("if exists (select * from sys.objects where name = 'Product_JDBC_Sample')
        + 'drop table Product_JDBC_Sample'");

    String sql = "CREATE TABLE [Product_JDBC_Sample] (" // Table N
        + "[ProductID] [int] IDENTITY(1,1) NOT NULL," // Attribute 1
        + "[Name] [varchar](30) NOT NULL," // Attribute 2
    );
    stmt.execute(sql);

    sql = "INSERT Product_JDBC_Sample VALUES ('Adjustable Time', 'AR-5381')";
    stmt.execute(sql);

    sql = "INSERT Product_JDBC_Sample VALUES ('ML Bottom Bracket', 'BB-8107')";
    stmt.execute(sql);
}
```

 IIT Madras  
RSC Campus

### JDBC: Example (3)

**Module 33**  
Partha Pratim Das  
Objectives & Outline  
SQL and Native Language  
ODBC  
Example: Python  
JDBC  
**Example: Java**  
Bridge  
Embedded SQL  
Example: C  
Example: Java  
Module Summary

```
private static void displayRow(String title, ResultSet rs) throws SQLException {
    System.out.println(title);
    while (rs.next()) { // Iterate on Table("ProductID", "Name")
        System.out.println(rs.getString("ProductID") + " : " + rs.getString("Name"));
    }
}

private static void createTable(Statement stmt) throws SQLException {
    stmt.execute("if exists (select * from sys.objects where name = 'Product_JDBC_Sample')
        + 'drop table Product_JDBC_Sample'");

    String sql = "CREATE TABLE [Product_JDBC_Sample] (" // Table N
        + "[ProductID] [int] IDENTITY(1,1) NOT NULL," // Attribute 1
        + "[Name] [varchar](30) NOT NULL," // Attribute 2
    );
    stmt.execute(sql);

    sql = "INSERT Product_JDBC_Sample VALUES ('Adjustable Time', 'AR-5381')";
    stmt.execute(sql);

    sql = "INSERT Product_JDBC_Sample VALUES ('ML Bottom Bracket', 'BB-8107')";
    stmt.execute(sql);
}
```

**JDBC: Example (3)**

```

private static void displayRow(String title, ResultSet rs) throws SQLException {
    System.out.println(title);
    while (rs.next()) { // Iterate on Table("ProductID", "Name")
        System.out.println(rs.getString("ProductID") + " : " + rs.getString("Name"));
    }
}

private static void createTable(Statement stmt) throws SQLException {
    stmt.execute("if exists (select * from sys.objects where name = 'Product_JDBC_Sample')");
    stmt.execute("drop table Product_JDBC_Sample");

    String sql = "CREATE TABLE [Product_JDBC_Sample] (" // Table N
        + "[ProductID] [int] IDENTITY(1,1) NOT NULL," // Attribute 1
        + "[Name] [varchar](30) NOT NULL," // Attribute 2
        + ")";

    stmt.execute(sql);

    sql = "INSERT Product_JDBC_Sample VALUES ('Adjustable Time', 'AR-5381')";
    stmt.execute(sql);

    sql = "INSERT Product_JDBC_Sample VALUES ('ML Bottom Bracket', 'BB-8107')";
    stmt.execute(sql);
}

```

So, here is create table, private static void create table which takes a statement. So, it has got the statement using which you will do the creation. So, statement has an execute which executes any SQL command. So, you have given a command, I mean, we want to create a table. So, the first thing I want to ensure that the table does not exist. So, I do a select, star in the sys subject, sys subject is what, sys object is a collection of the data dictionary, where all table names and all those are kept. So, if that selection returns anything, then the table exists. So, if it exists then we will drop the table. If it does not, then we will do nothing.

So, now I formed the query for creating the table. This is create table the name, the first attribute with all the, this is just pure plain and simple, pure SQL except this notation for the name. Everything else is exactly the same. Maybe the way you write the types are little different, but it exactly corresponds to SQL. So, I create a table with two attributes one and two. And then the statement I execute. The table is created. And then I keep on making a statement take a string to insert a pair of values as a product one execute, so first insert done, second insert done, third insert done. So, after I have executed this method, obviously, I will have three, the table having three products into it.

(Refer Slide Time: 19:17)

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class RetrieveResultSet {

    public static void main(String[] args) {
        // Create a variable for the connection string.
        String connectionUrl = "jdbc:sqlserver://<server>:<port>;databaseName=Adve
        connectionUrl += "user=<user>; password=<password>";

        try (Connection con = DriverManager.getConnection(connectionUrl);
            Statement stmt = con.createStatement();) {
            createTable(stmt);
            String SQL = "SELECT * FROM Production.Product;";
            ResultSet rs = stmt.executeQuery(SQL);
            displayRow("PRODUCTS", rs);
        }
    }
}
```

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class RetrieveResultSet {

    public static void main(String[] args) {
        // Create a variable for the connection string.
        String connectionUrl = "jdbc:sqlserver://<server>:<port>;databaseName=Adve
        connectionUrl += "user=<user>; password=<password>";

        try (Connection con = DriverManager.getConnection(connectionUrl);
            Statement stmt = con.createStatement();) {
            createTable(stmt);
            String SQL = "SELECT * FROM Production.Product;";
            ResultSet rs = stmt.executeQuery(SQL);
            displayRow("PRODUCTS", rs);
        }
    }
}
```

So, let me get back here. So, I have done create table. Table is ready. Now, I make the query string, select star from the table and I define a result set. I said, the result will be in terms of a table. So, that is called the result set. So, this is this actually is like a table which we will be able to iterate and print. And then I do statement.execute query. On the statement I execute the query like this. I could have done statement execute within that, any of that is possible. Then I do display row. So, I have got the result set, now I have to show that, display that, create the HTML and so on.

(Refer Slide Time: 20:13)

**JDBC: Example (3)**

```
private static void displayRow(String title, ResultSet rs) throws SQLException {
    System.out.println(title);
    while (rs.next()) { // Iterate on Table("ProductID", "Name")
        System.out.println(rs.getString("ProductID") + " : " + rs.getString("Name"));
    }
}

private static void createTable(Statement stmt) throws SQLException {
    stmt.execute("if exists (select * from sys.objects where name = 'Product_JDBC_Sample')
        + 'drop table Product_JDBC_Sample'");

    String sql = "CREATE TABLE [Product_JDBC_Sample] (" // Table N
        + "[ProductID] [int] IDENTITY(1,1) NOT NULL," // Attribute
        + "[Name] [varchar](30) NOT NULL,)" // Attribute
    stmt.execute(sql);

    sql = "INSERT Product_JDBC_Sample VALUES ('Adjustable Time', 'AR-5381')";
    stmt.execute(sql);

    sql = "INSERT Product_JDBC_Sample VALUES ('ML Bottom Bracket', 'BB-8107')";
    stmt.execute(sql);
}
```

**JDBC: Example (3)**

```
private static void displayRow(String title, ResultSet rs) throws SQLException {
    System.out.println(title);
    while (rs.next()) { // Iterate on Table("ProductID", "Name")
        System.out.println(rs.getString("ProductID") + " : " + rs.getString("Name"));
    }
}

private static void createTable(Statement stmt) throws SQLException {
    stmt.execute("if exists (select * from sys.objects where name = 'Product_JDBC_Sample')
        + 'drop table Product_JDBC_Sample'");

    String sql = "CREATE TABLE [Product_JDBC_Sample] (" // Table N
        + "[ProductID] [int] IDENTITY(1,1) NOT NULL," // Attribute
        + "[Name] [varchar](30) NOT NULL,)" // Attribute
    stmt.execute(sql);

    sql = "INSERT Product_JDBC_Sample VALUES ('Adjustable Time', 'AR-5381')";
    stmt.execute(sql);

    sql = "INSERT Product_JDBC_Sample VALUES ('ML Bottom Bracket', 'BB-8107')";
    stmt.execute(sql);
}
```

So, let me again, go to the next slide. So, what I have displayRow which takes a title and result set. So, the title is the title of the table. So, I print that title, which is simple. Then what I am doing is on the, I iterate on this table with the, for the product ID rs dot next, so this is the iterator. And at every iteration value, I get the string corresponding to product ID and the name. This is the two things I had done product ID and name. So, that is the two columns that have come. So, I take two of those values from the row print it, take the next two print it, take the next two print it and I am done. So, this is how very easily using JDBC a simple Java application with database can be created.

(Refer Slide Time: 21:06)

**Connectionist Bridge Configurations**

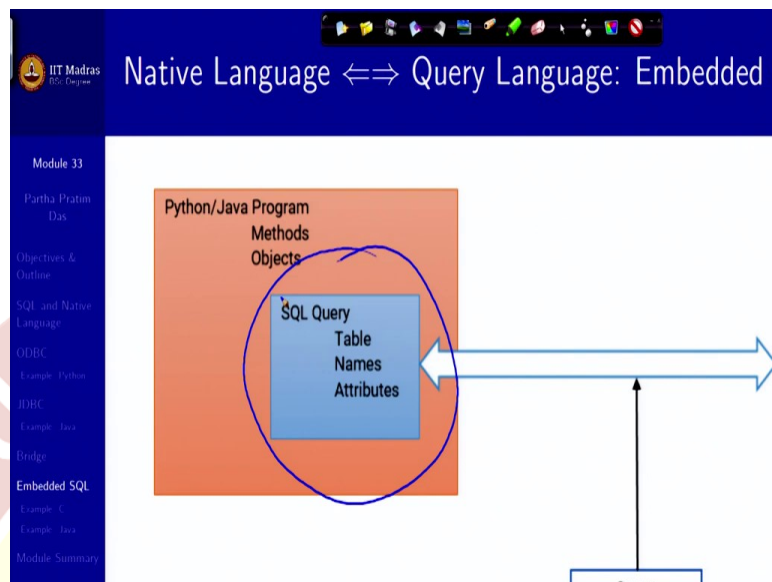
A **bridge** is a special kind of driver that uses another driver-based technique.

- This driver translates *source function-calls* into *target function-calls*
- Programmers usually use such a bridge when they lack a *source driver* database but have access to a *target driver*
- Common bridges are:
  - **ODBC-to-JDBC (ODBC-JDBC) bridges:** An ODBC-JDBC bridge consists of which uses the services of a JDBC driver to connect to a database. *Examples:* ODBC-JDBC Bridge, SequeLink ODBC-JDBC Bridge
  - **JDBC-to-ODBC (JDBC-ODBC) bridges:** A JDBC-ODBC bridge consists of employs an ODBC driver to connect to a target database. *Examples:* OpenLink Bridge, SequeLink JDBC-ODBC Bridge
  - **OLE DB-to-ODBC bridges:** An OLE DB-ODBC bridge consists of an OLE DB which uses the services of an ODBC driver to connect to a target database. This proxy uses the services of an ODBC driver to connect to a target database. *Examples:* OpenLink OLEDB-ODBC Bridge, SequeLink OLEDB-ODBC Bridge

Now, there are often questions that certain databases may support ODBC, but you may want to use Java JDBC with that or vice versa. So, there are certain bridge drivers which bridge between two of these drivers. So, ODBC to JDBC bridge, it consists of a ODBC driver, which uses a JDBC driver to connect to a database. So, that your application program is written in ODBC, but the database is actually supporting JDBC.

You can still using this bridge driver you can do that. You can do the reverse also. You can do it from OLEDB to ODBC, you can do ADO dot NET to ODBC. So, basically putting all this together, you have immense connectionist power to use any of these drivers from any languages and connect to any database with any kind of support in terms of the connectivity to the application program. So, that was the connectionist view.

(Refer Slide Time: 22:24)



Next what we see is the embedding view, which is where what you do you do a different thing. You put the SQL query as a part of your source language and let the whole thing handle, be handled in the query processing itself.

(Refer Slide Time: 22:40)

The slide is titled "Embedded SQL" and features a sidebar on the left with the same module list as the previous slide. The main content area contains a list of bullet points:

- The SQL standard defines embedding of SQL in a variety of program such as C, C++, Java, FORTRAN, and PL/1
- A language to which SQL queries are embedded is referred to as a **host language**. The SQL structures permitted in the host language comprise **embedded SQL**.
- The basic form of these languages follows that of the System R embedded SQL.
- **EXEC SQL** (or similar alternate like **#sql**) statement is used to identify a request to the pre-processor

A blue box highlights the text: **EXEC SQL <embedded SQL statement >;**

Note: this varies by language.

- In some languages, like COBOL, the semicolon is replaced with E

So, for that you need, again, what do you need, you need to say that you execute a statement. So, you have a host language say Python or Java or C++ and in that you want some unique way to say that this part is SQL. So, you use a tag like EXEC SQL or some systems use #sql and

anything which is SQL you put inside that. So, the general structure is like that, EXEC SQL then you have an SQL statement. That is one part.

(Refer Slide Time: 23:22)

**Embedded SQL (2)**

- Before executing any SQL statements, the program must first connect to the server. This is done using:  
EXEC-SQL connect to server user user-name using password;  
Here, server identifies the server to which a connection is to be established.
- Variables of the host language can be used within embedded SQL statements preceded by a colon (:) to distinguish from SQL variables (for example).
- Variables used as above must be declared within DECLARE section, and the syntax for declaring the variables, however, follows the usual host language syntax.

```
EXEC-SQL BEGIN DECLARE SECTION
int credit-amount ;
```

**Embedded SQL (2)**

- Before executing any SQL statements, the program must first connect to the server. This is done using:  
EXEC-SQL connect to server user user-name using password;  
Here, server identifies the server to which a connection is to be established.
- Variables of the host language can be used within embedded SQL statements preceded by a colon (:) to distinguish from SQL variables (for example).
- Variables used as above must be declared within DECLARE section, and the syntax for declaring the variables, however, follows the usual host language syntax.

```
EXEC-SQL BEGIN DECLARE SECTION
int credit-amount ;
```

So, what will you need to do along with this? You will need to connect the first step. So, this is straightforward. This is straightforward. EXEC SQL connect to server name, user, user name using password. This will give you the connection. For multiple SQL commands, you write a section and you write the commands, SQL commands within that.

(Refer Slide Time: 24:52)



IIT Madras  
B.Tech. Degree

Module 33

Patha Pratham  
Das

Objectives &  
Outline

SQL and Native  
Language

DBMS

Example: Python

DBMS

Example: Java

DBMS

Example: JavaScript

Module Summary

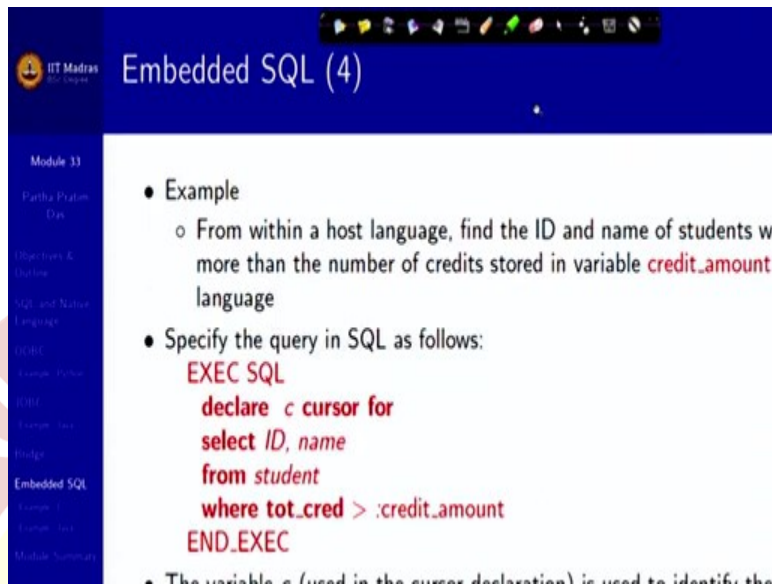
# Embedded SQL (3)

- To write an embedded SQL query, we use the declare c cursor for <SQL query> statement. The variable c is used to identify the query
- Example:
  - From within a host language, find the ID and name of students with more than the number of credits stored in variable **credit\_amount** language
  - Specify the query in SQL as follows:

```
EXEC SQL
declare c cursor for
select ID, name
from student
```

Now, we have seen that in the connectionist we need a way to take the table out. So, what we did we used iterators. So, we need a similar thing here. So, what is supported for embedding is defining declaring cursor. So, you declare c as a cursor for an SQL query. So, this is declare c as a cursor for this query, which means the result of this query will be available in this cursor c and then you can iterate on that cursor to get the values out. You can see that in the query we have used a host language variable value by using a colon before it so that it becomes a part of the SQL. Structurally very simple.

(Refer Slide Time: 25:56)



**Embedded SQL (4)**

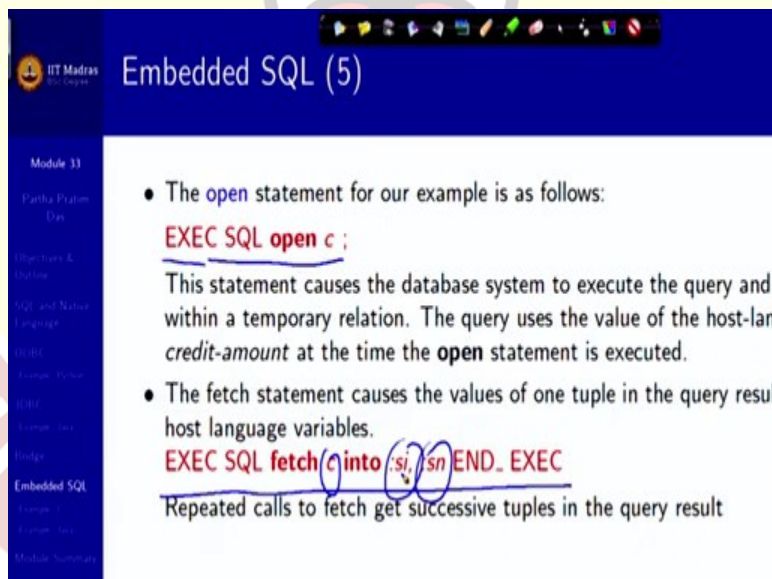
- Module 11
  - Partha Pratim Das
  - Objectives & Outline
  - SQL and Native Language
  - ODBC
  - Example: Python
  - ODBC
  - Example: Java
  - Bridge
  - Embedded SQL
  - Example: C
  - Example: Java
  - Module Summary

- Example
  - From within a host language, find the ID and name of students with more than the number of credits stored in variable `credit_amount` language
- Specify the query in SQL as follows:  
**EXEC SQL**  
**declare c cursor for**  
**select ID, name**  
**from student**  
**where tot\_cred > :credit\_amount**  
**END EXEC**

• The variable `c` (used in the cursor declaration) is used to identify the

This is we have already discussed.

(Refer Slide Time: 26:01)



**Embedded SQL (5)**

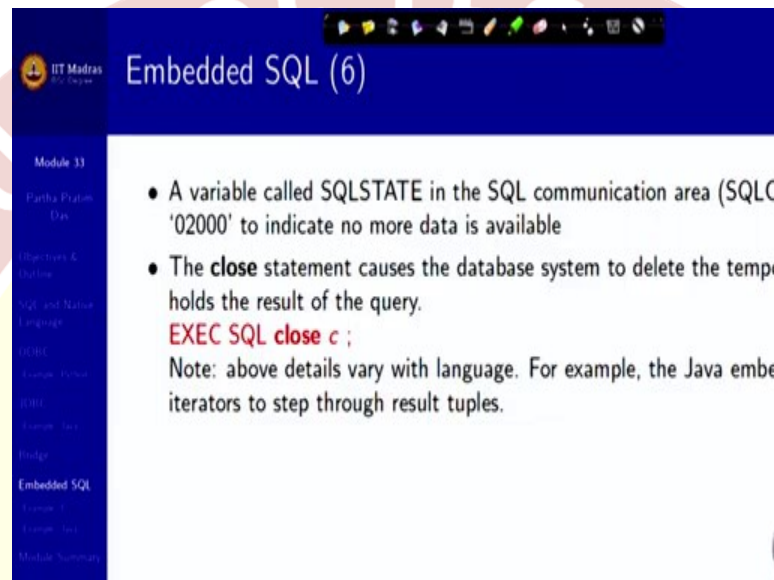
- Module 11
  - Partha Pratim Das
  - Objectives & Outline
  - SQL and Native Language
  - ODBC
  - Example: Python
  - ODBC
  - Example: Java
  - Bridge
  - Embedded SQL
  - Example: C
  - Example: Java
  - Module Summary

- The `open` statement for our example is as follows:  
**EXEC SQL open c ;**  
This statement causes the database system to execute the query and within a temporary relation. The query uses the value of the host-language `credit-amount` at the time the `open` statement is executed.
- The fetch statement causes the values of one tuple in the query result host language variables.  
**EXEC SQL fetch c into :si, :sn END EXEC**  
Repeated calls to fetch get successive tuples in the query result

Now, after a connection, you open a cursor, you have defined a cursor, you open a cursor. So, cursor needs to be open so that it is available for taking your data. And then you fetch `c` into, `c` is a cursor, into what are these, these are obviously host language variables. So, from the cursor, you take the values to the specific host language variables.

Keep on doing that, repeatedly keep on calling the fetch in the SQL and you will keep on getting the subsequent tuples. Your iteration happens. So, it is declaring a cursor, opening it, executing the query and then doing fetch on the cursor repeatedly and every time you get the entire tuple you need to have placeholders for host language variables, which will get those values.

(Refer Slide Time: 27:13)

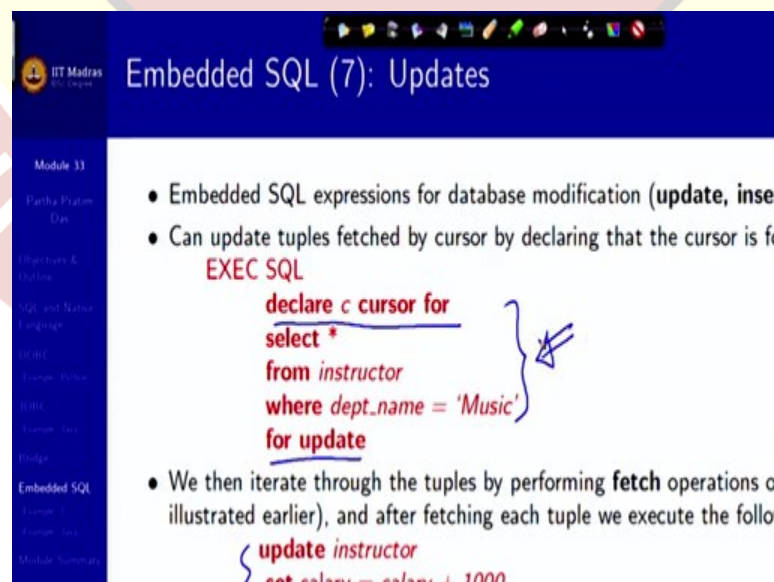


**Embedded SQL (6)**

- A variable called SQLSTATE in the SQL communication area (SQLCA) with value '02000' to indicate no more data is available
- The **close** statement causes the database system to delete the temporary result set and holds the result of the query.  
**EXEC SQL close c ;**  
Note: above details vary with language. For example, the Java embedded SQL uses iterators to step through result tuples.

Finally, when you are done, you close the cursor. It is no more available.

(Refer Slide Time: 27:23)

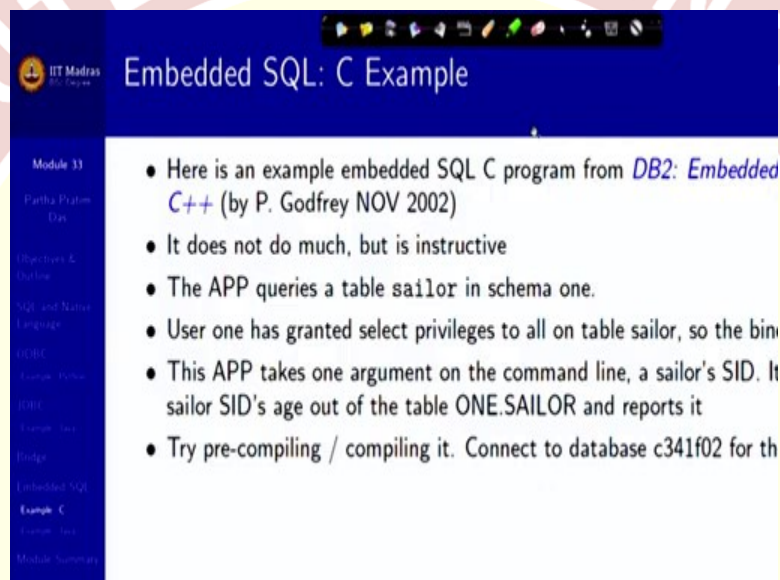


**Embedded SQL (7): Updates**

- Embedded SQL expressions for database modification (**update, insert, delete**)
- Can update tuples fetched by cursor by declaring that the cursor is for **update**  
**EXEC SQL**  
**declare c cursor for**  
**select \***  
**from instructor**  
**where dept\_name = 'Music'**  
**for update**  
We then iterate through the tuples by performing **fetch** operations (as illustrated earlier), and after fetching each tuple we execute the following:  
**update instructor**  
**set salary = salary + 1000**

So, some you can do this for update as well. You can give a, take a cursor for update and then update using that cursor. So, here you are declaring a cursor. This is your query. You get the list of instructors. The cursor is for update you have set. And then you update instructor. This is same as before. You say the where the current of c. So, instead of actually giving you a separate relation to update from, you are giving a temporary table which is this cursor as a result of this. So, these are the different variants that you get.

(Refer Slide Time: 28:14)



The screenshot shows a presentation slide from IIT Madras titled "Embedded SQL: C Example". The slide content is as follows:

- Here is an example embedded SQL C program from *DB2: Embedded C++* (by P. Godfrey NOV 2002)
- It does not do much, but is instructive
- The APP queries a table sailor in schema one.
- User one has granted select privileges to all on table sailor, so the bin
- This APP takes one argument on the command line, a sailor's SID. It sailor SID's age out of the table ONE.SAILOR and reports it
- Try pre-compiling / compiling it. Connect to database c341f02 for th

So, time for a full example. So, here there is a sailor table. This is a C example. So, this is a sailor table which from where you can give the ID and get the sailor's age. That is a very simple.

(Refer Slide Time: 28:33)

**Module 33**  
Partha Pratim Das  
Objectives & Outline  
SQL and Native Language  
ODBC  
Example: Python  
JDBC  
Example: Java  
Bridge  
Embedded SQL  
Example: C  
Example: Java  
Module Summary

### Embedded SQL: C Example (2)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sqlenv.h>
#include <sqlcodes.h>
#include <sys/time.h>

#define EXIT 0
#define NOEXIT 1

EXEC SQL INCLUDE SQLCA; // Include DB2's SQL error reporting facility.

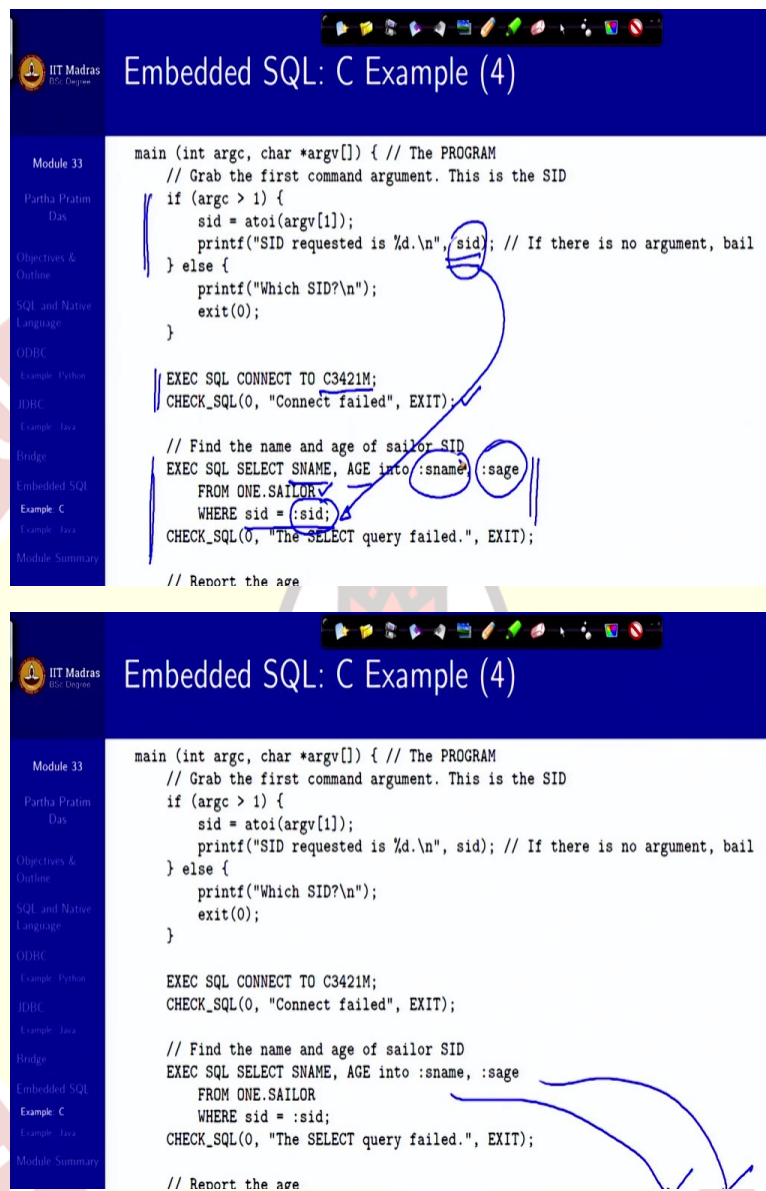
EXEC SQL BEGIN DECLARE SECTION; // Declare the SQL interface variables.
    short sage, sid; char sname[16];
EXEC SQL END DECLARE SECTION;

// Declare variables to be used in the following C program
char msg[1025]; int rc, errcount;

// This macro prints the message in the SQLCA if the return code is 0 and the SQLC
```

So, let us see what are the embedding requirements. So, these two are for, are headers for embedding SQL. This includes the error reporting, it says that the error reporting can be done. This is declarations section where you are declaring host language variables to be used in SQL sage sid, sname. This is simple programming variables. Then you have a macro for printing a message which the macro prints a message in SQLCA if the return code is 0 and is error handling. Let us look at the meet of the thing.

(Refer Slide Time: 29:28)



**Embedded SQL: C Example (4)**

```
main (int argc, char *argv[]) { // The PROGRAM
    // Grab the first command argument. This is the SID
    if (argc > 1) {
        sid = atoi(argv[1]);
        printf("SID requested is %d.\n", sid); // If there is no argument, bail
    } else {
        printf("Which SID?\n");
        exit(0);
    }

    EXEC SQL CONNECT TO C3421M;
    CHECK_SQL(0, "Connect failed", EXIT);

    // Find the name and age of sailor SID
    EXEC SQL SELECT SNAME, AGE INTO :sname :sage
    FROM ONE.SAILOR
    WHERE sid = :sid;
    CHECK_SQL(0, "The SELECT query failed.", EXIT);

    // Report the age
```

**Embedded SQL: C Example (4)**

```
main (int argc, char *argv[]) { // The PROGRAM
    // Grab the first command argument. This is the SID
    if (argc > 1) {
        sid = atoi(argv[1]);
        printf("SID requested is %d.\n", sid); // If there is no argument, bail
    } else {
        printf("Which SID?\n");
        exit(0);
    }

    EXEC SQL CONNECT TO C3421M;
    CHECK_SQL(0, "Connect failed", EXIT);

    // Find the name and age of sailor SID
    EXEC SQL SELECT SNAME, AGE INTO :sname :sage
    FROM ONE.SAILOR
    WHERE sid = :sid;
    CHECK_SQL(0, "The SELECT query failed.", EXIT);

    // Report the age
```

Now, let me go to the main first. So, what do you have in the main, you take a main as argc greater than 1. So, it checks that whether it is got a command line input. If it is there, it will go, otherwise it will not work. If it is there, then that is the sailor ID for whom I want the sailor age. So, I connect. This is the database name hard coded here for that example, some database name. This is the check, the macro to check for error. If there is error, it will exit. Then I have EXEC SQL, the actual query. So, what is sid is colon sid. This is the this sid, host language variable, so that ID will convey there. This is a table and I have got sname, sage into colon sname colon sage, which are also host language variable.

Here, I am not getting into so much of cursor and all because I know there will be only one variable. So, then you, once this is done, then you can report, because you have the sname, you have the sage. So, this goes here, this goes here. If there is error, you reset the connection. So, this is a sample scheme of actually doing this.

(Refer Slide Time: 31:09)

The screenshot shows a presentation slide titled "Embedded SQL: C Example (5)". On the left is a blue sidebar with a table of contents including "Module 31", "Partha Pratan Das", "Objectives & Outline", "SQL and Native Language", "SQL", "Example: C", and "Module Summary". The main content area has a blue header with the title. Below the header, there is a bullet point: "The instance of the table sailor:". This is followed by a table with columns SNAME, SID, RATING, and AGE. The table contains five rows: yuppy (SID 22, RATING 1, AGE 20), lubber (SID 31, RATING 1, AGE 25), guppy (SID 44, RATING 2, AGE 31), and rusty (SID 58, RATING 3, AGE 47). A blue bracket on the right groups the last three rows. Below the table, another bullet point says: "If the name of the executable is sage, and if you ask: % sage 44". The "44" is circled in blue. A final bullet point states: "The output should be: SID requested is 44.", with a blue checkmark next to it.

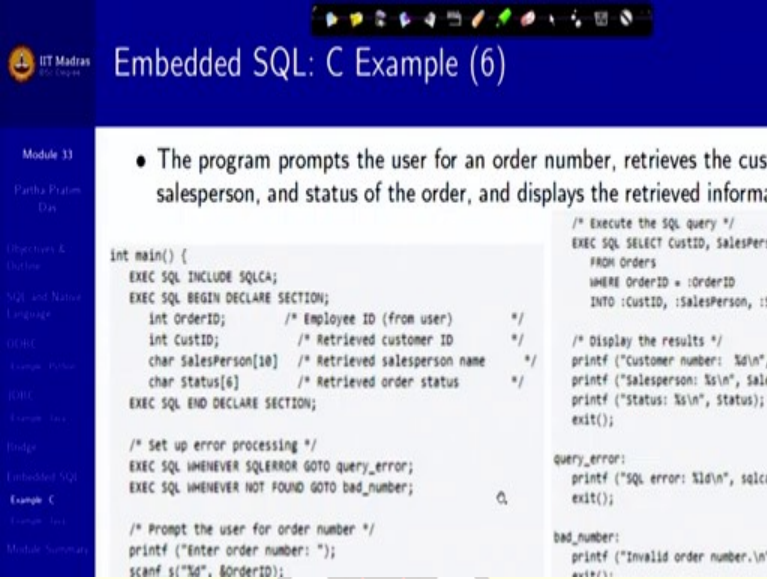
| SNAME        | SID       | RATING | AGE |
|--------------|-----------|--------|-----|
| yuppy        | 22        | 1      | 20  |
| lubber       | 31        | 1      | 25  |
| <u>guppy</u> | <u>44</u> | 2      | 31  |
| rusty        | 58        | 3      | 47  |

• If the name of the executable is sage, and if you ask:  
% sage 44

• The output should be:  
SID requested is 44. ✓

So, there is a, if this is just to tell you what the result should be, if this is the instance and if your input is assuming sage is application name is 44, that is sid 44 then you will get, should get this sid request is 44. This is guppy. So, guppy's age is 31 executed successfully. So, this is an example of how you can do it through embedding.

(Refer Slide Time: 31:43)



**Embedded SQL: C Example (6)**

- The program prompts the user for an order number, retrieves the customer, salesperson, and status of the order, and displays the retrieved information:

```
int main() {
    EXEC SQL INCLUDE SQLCA;
    EXEC SQL BEGIN DECLARE SECTION;
        int OrderID;      /* Employee ID (from user) */
        int CustID;       /* Retrieved customer ID */
        char SalesPerson[10] /* Retrieved salesperson name */
        char Status[6]     /* Retrieved order status */
    EXEC SQL END DECLARE SECTION;

    /* Set up error processing */
    EXEC SQL WHENEVER SQLERROR GOTO query_error;
    EXEC SQL WHENEVER NOT FOUND GOTO bad_number;

    /* Prompt the user for order number */
    printf ("Enter order number: ");
    scanf ("%d", &OrderID);

    /* Execute the SQL query */
    EXEC SQL SELECT CustID, SalesPerson, Status
    FROM Orders
    WHERE OrderID = :OrderID
    INTO :CustID, :SalesPerson, :Status;

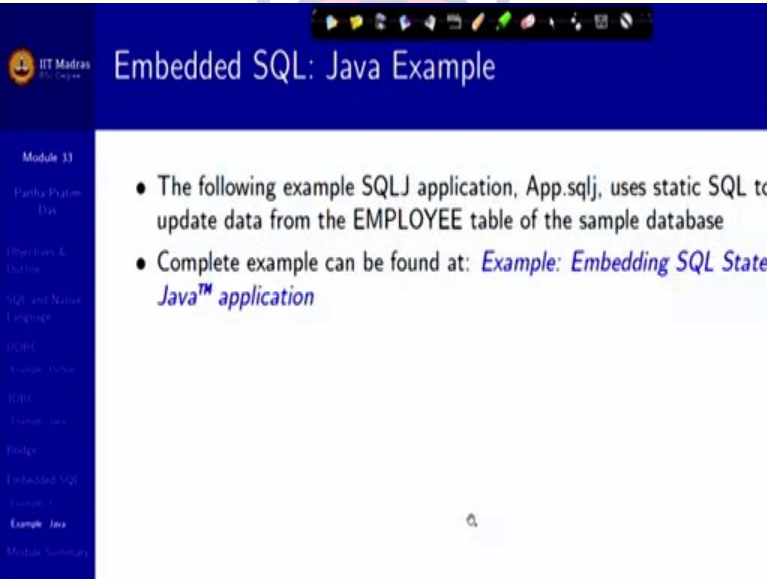
    /* Display the results */
    printf ("Customer number: %d\n", CustID);
    printf ("Salesperson: %s\n", SalesPerson);
    printf ("Status: %s\n", Status);
    exit();

query_error:
    printf ("SQL error: %ld\n", sqlca.sqlcode);
    exit();

bad_number:
    printf ("Invalid order number.\n");
    exit();
}
```

Just I have given another example in C, which you should read and understand it better.

(Refer Slide Time: 31:53)

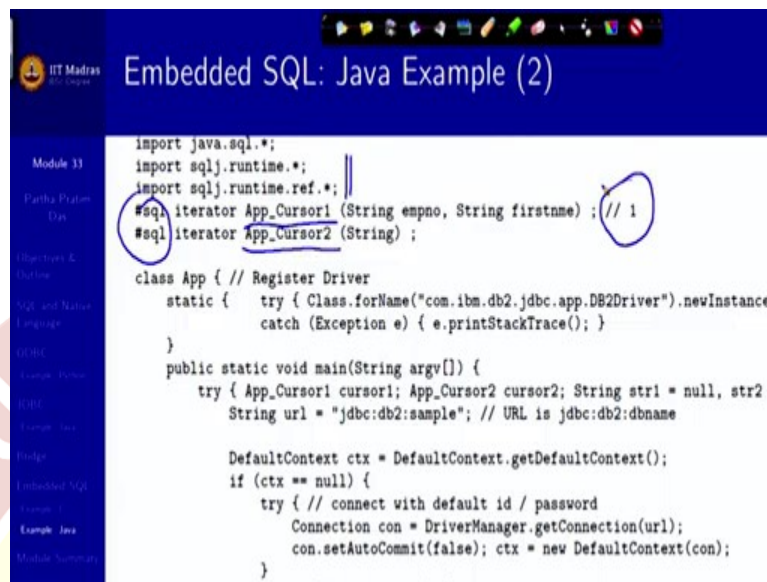


**Embedded SQL: Java Example**

- The following example SQLJ application, App.sqlj, uses static SQL to update data from the EMPLOYEE table of the sample database
- Complete example can be found at: [Example: Embedding SQL State Java™ application](#)

I have also given an example in Java, which is little bit longish.

(Refer Slide Time: 32:00)



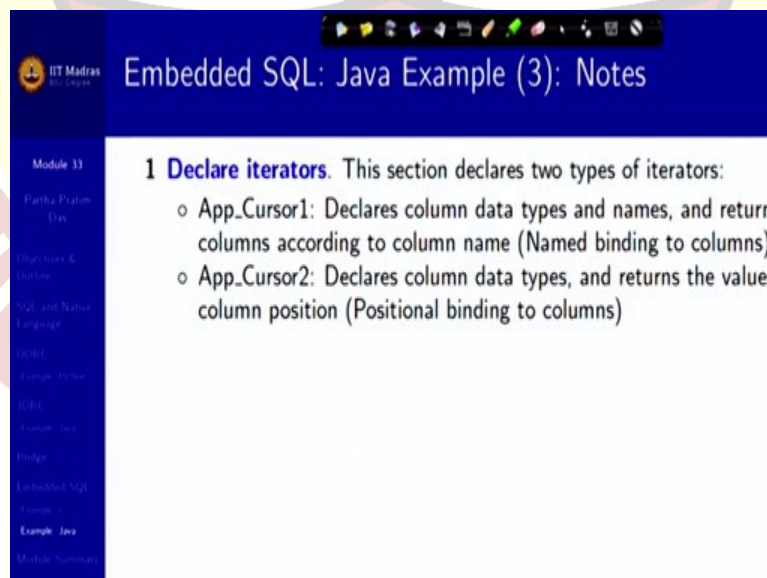
```
import java.sql.*;
import sqlj.runtime.*;
import sqlj.runtime.ref.*;
#sql iterator App_Cursor1 (String empno, String firstme) ; // 1
#sql iterator App_Cursor2 (String) ;

class App { // Register Driver
    static { try { Class.forName("com.ibm.db2.jdbc.app.DB2Driver").newInstance();
                catch (Exception e) { e.printStackTrace(); }
            }
    public static void main(String argv[]) {
        try { App_Cursor1 cursor1; App_Cursor2 cursor2; String str1 = null, str2 =
            String url = "jdbc:db2:sample"; // URL is jdbc:db2:dbname

            DefaultContext ctx = DefaultContext.getDefaultContext();
            if (ctx == null) {
                try { // connect with default id / password
                    Connection con = DriverManager.getConnection(url);
                    con.setAutoCommit(false); ctx = new DefaultContext(con);
                }
            }
        }
    }
}
```

But this is, this has more functionality in terms of actually having the two different types of cursors. And you can see that in this particular embedding framework, instead of EXEC SQL it is using #sql. In different places in this code, you will find that I have given markers, I mean, comment with number so that you can go to the next slide. So, one here is an application cursor.

(Refer Slide Time: 32:38)

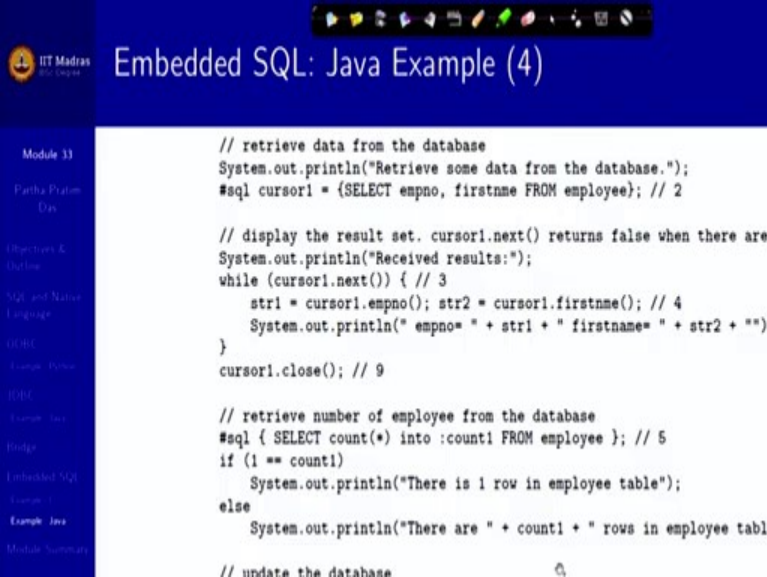


**1 Declare iterators.** This section declares two types of iterators:

- App\_Cursor1: Declares column data types and names, and returns columns according to column name (Named binding to columns)
- App\_Cursor2: Declares column data types, and returns the values column position (Positional binding to columns)

So, if you go to the next slide, you will find what is the explanation of what that step, in terms of embedded SQL is doing. In this way, it goes on.

(Refer Slide Time: 32:52)



**Module 33**  
Partha Pratim Das

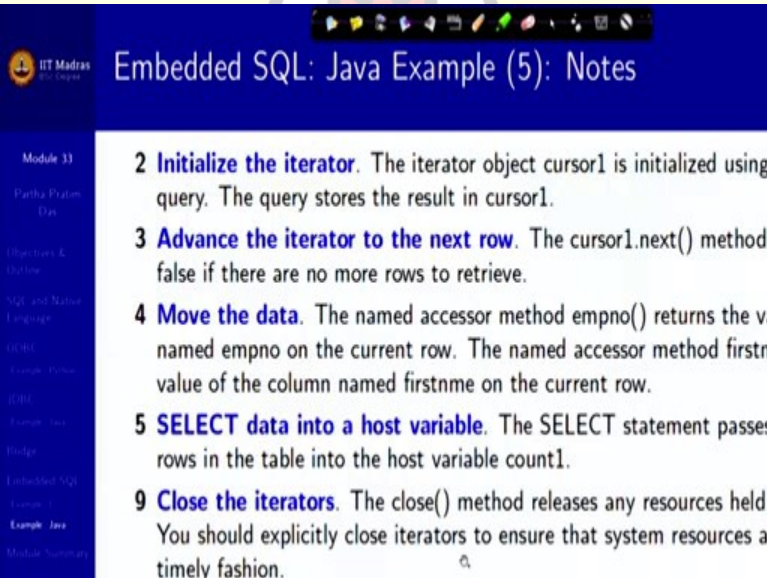
Objectives & Outline  
SQL and Native Language  
JDBC  
Example: Python  
JDBC  
Example: Java  
Bridge  
Embedded SQL  
Example: C  
Example: Java  
Module Summary

```
// retrieve data from the database
System.out.println("Retrieve some data from the database.");
#sql cursor1 = {SELECT empno, firstnme FROM employee}; // 2

// display the result set. cursor1.next() returns false when there are
System.out.println("Received results:");
while (cursor1.next()) { // 3
    str1 = cursor1.empno(); str2 = cursor1.firstnme(); // 4
    System.out.println(" empno= " + str1 + " firstname= " + str2 + "")
}
cursor1.close(); // 9

// retrieve number of employee from the database
#sql { SELECT count(*) into :count1 FROM employee }; // 5
if (1 == count1)
    System.out.println("There is 1 row in employee table");
else
    System.out.println("There are " + count1 + " rows in employee tabl

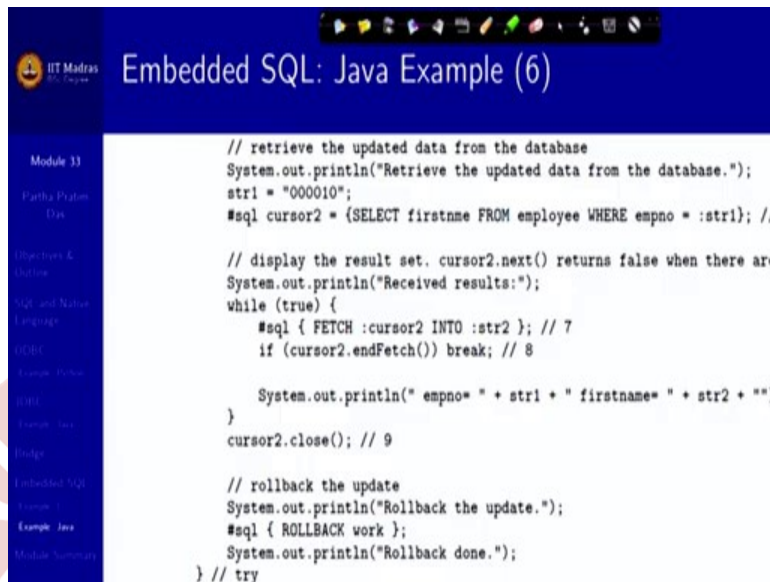
// update the database
```



**Module 33**  
Partha Pratim Das

Objectives & Outline  
SQL and Native Language  
JDBC  
Example: Python  
JDBC  
Example: Java  
Bridge  
Embedded SQL  
Example: C  
Example: Java  
Module Summary

- 2 Initialize the iterator.** The iterator object cursor1 is initialized using query. The query stores the result in cursor1.
- 3 Advance the iterator to the next row.** The cursor1.next() method false if there are no more rows to retrieve.
- 4 Move the data.** The named accessor method empno() returns the v. named empno on the current row. The named accessor method firstn value of the column named firstnme on the current row.
- 5 SELECT data into a host variable.** The SELECT statement passes rows in the table into the host variable count1.
- 9 Close the iterators.** The close() method releases any resources held You should explicitly close iterators to ensure that system resources a timely fashion.



**Module 33**  
Partha Pratim Das

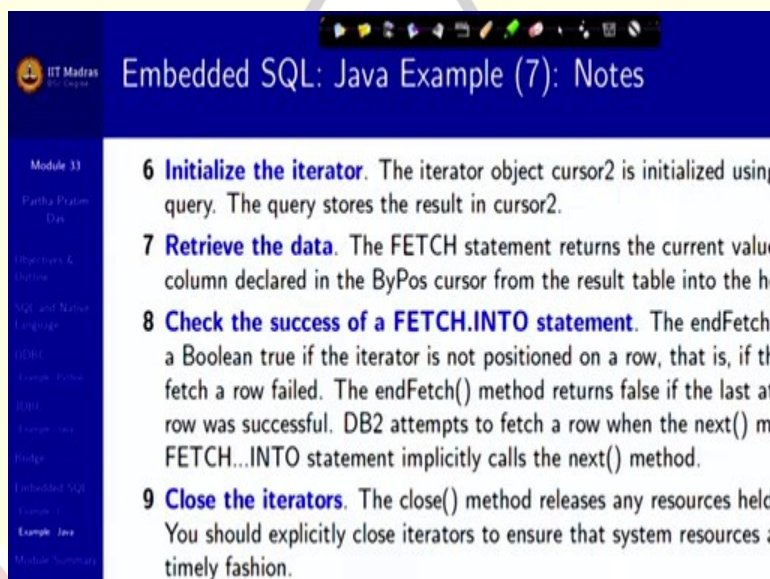
Objectives & Outline  
SQL and Native Language  
JDBC  
Example: Python  
JDBC  
Example: Java  
Bridge  
Embedded SQL  
Example: C  
Example: Java  
Module Summary

```
// retrieve the updated data from the database
System.out.println("Retrieve the updated data from the database.");
str1 = "000010";
#sql cursor2 = {SELECT firstnme FROM employee WHERE empno = :str1}; //

// display the result set. cursor2.next() returns false when there are
System.out.println("Received results:");
while (true) {
    #sql { FETCH :cursor2 INTO :str2 }; // 7
    if (cursor2.endFetch()) break; // 8

    System.out.println(" empno= " + str1 + " firstname= " + str2 + "")
}
cursor2.close(); // 9

// rollback the update
System.out.println("Rollback the update.");
#sql { ROLLBACK work };
System.out.println("Rollback done.");
} // try
```



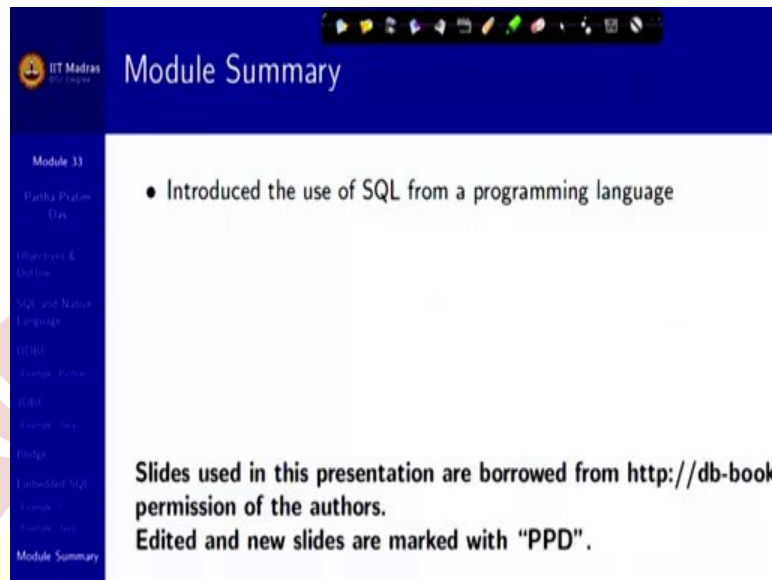
**Module 33**  
Partha Pratim Das

Objectives & Outline  
SQL and Native Language  
JDBC  
Example: Python  
JDBC  
Example: Java  
Bridge  
Embedded SQL  
Example: C  
Example: Java  
Module Summary

- 6 Initialize the iterator.** The iterator object cursor2 is initialized using query. The query stores the result in cursor2.
- 7 Retrieve the data.** The FETCH statement returns the current value column declared in the ByPos cursor from the result table into the hc
- 8 Check the success of a FETCH.INTO statement.** The endFetch() returns a Boolean true if the iterator is not positioned on a row, that is, if the fetch a row failed. The endFetch() method returns false if the last at row was successful. DB2 attempts to fetch a row when the next() or FETCH...INTO statement implicitly calls the next() method.
- 9 Close the iterators.** The close() method releases any resources held. You should explicitly close iterators to ensure that system resources are released in a timely fashion.

It is little long to discuss in the lecture, but I have included it so that you can conveniently read through the code and look at the comments and get a complete feel of an application. This is just for a variant I have used Java here. So, this is the hole and with the final notes.

(Refer Slide Time: 33:12)



**IIT Madras**  
Module Summary

**Module 11**  
Partha Pratim Das

- Introduced the use of SQL from a programming language

Objectives & Outline  
SQL and Native Language  
ODBC  
Connection Pooling  
JDBC  
Connection Pooling  
Bridge  
Embedded SQL  
Connection Pooling  
Connection Pooling  
Module Summary

Slides used in this presentation are borrowed from <http://db-book> permission of the authors.  
Edited and new slides are marked with "PPD".

So, that brings us to the end of this module. We have discussed here how to access a database through SQL from a host programming language through connectionist or embedded styles. Thank you very much for your attention. We will do more on this in the next module.